Does it fit?

Size-constrained agglomerative clustering · legality, the target and three endings

Learning Clustering · original author GateUltra (Issue #3)

The problem

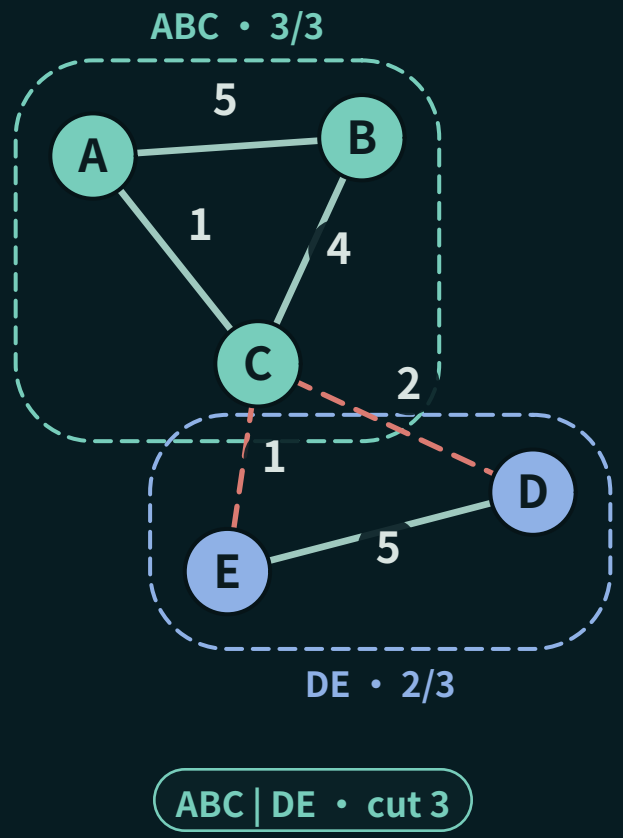
- Input: a graph, node sizes s(v), a capacity C, a target K
- Rule: filter for legality (merged size ⩽ C), then take the highest affinity
- K never overrides the capacity: no over-capacity merges
- A single node over capacity: the input is invalid
- Output four separate items: count, legal, K reached, stop reason

Report item	Capacity 2, K = 2
count	3
capacityCompliant	yes
targetReached	no
stopReason	capacity-blocked

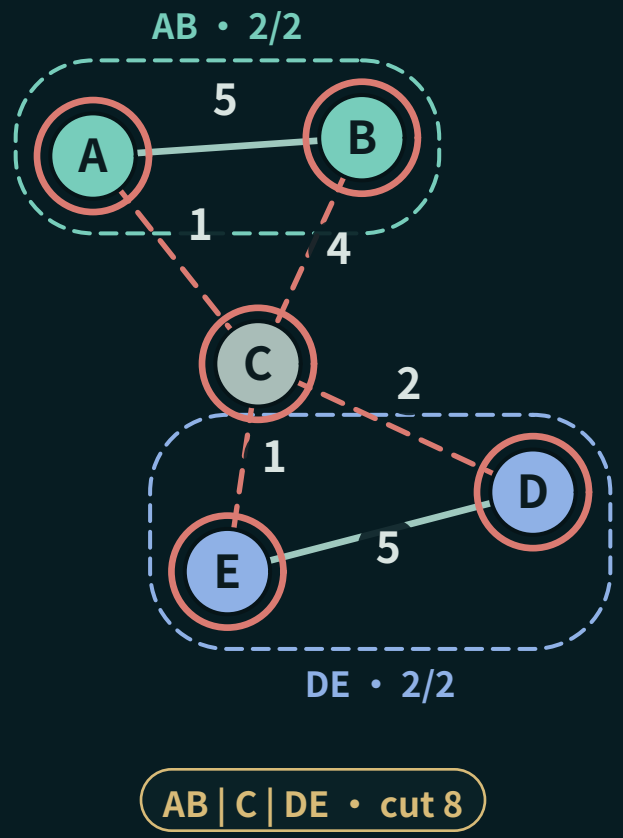
One graph, two capacities

- Capacity 3: AB+C has size 3, legal, giving ABC | DE, cut 3
- Capacity 2: AB+C is blocked, so D–E instead
- After that AB+C and C+DE are both 3: all blocked
- Capacity 2 result: AB | C | DE, cut 8, K not reached

Capacity 3: K reached



Capacity 2: K not reached



Impossible: a lower-bound proof

- Lower bound: at least "total size ÷ C, rounded up" clusters
- G01: 5 ÷ 2 rounded up = 3 > K = 2
- No algorithm can do it: greedy made no mistake by stopping
- A bound can only prove "impossible", never "possible"

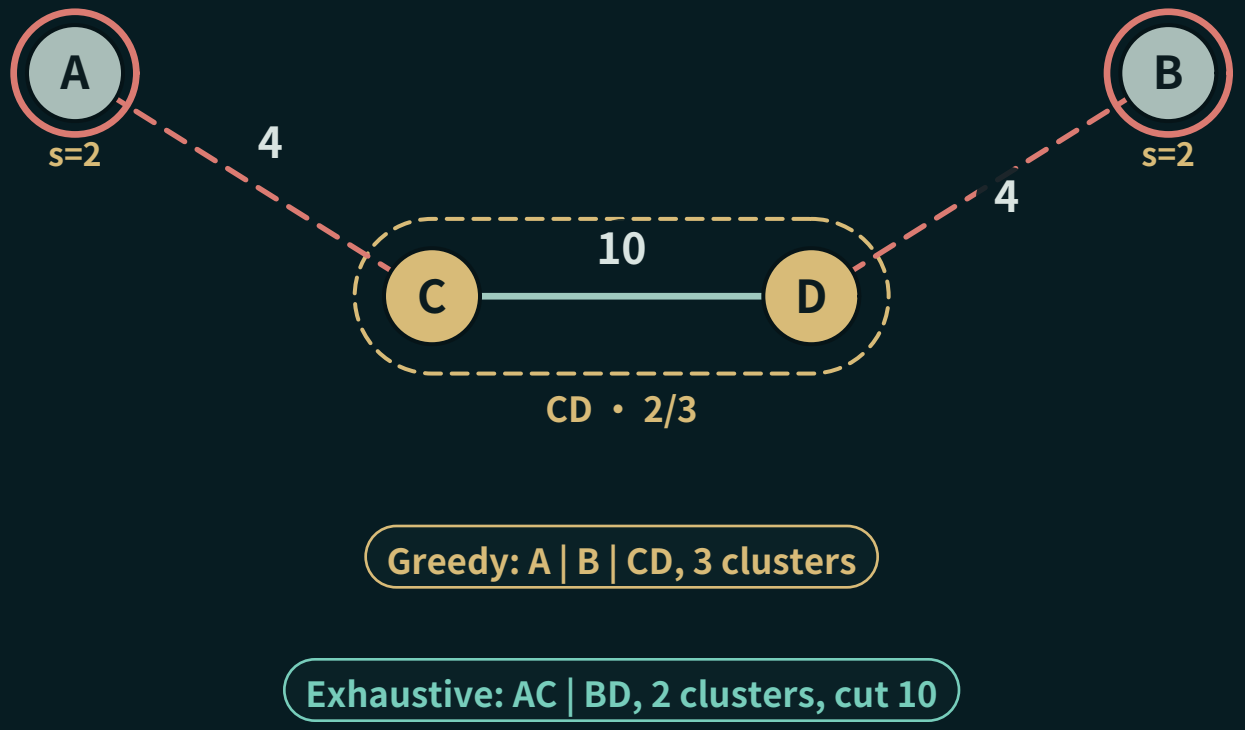


$$\lceil 5 \div 2 \rceil = 3 > K = 2$$

at least 3 clusters: K = 2 is impossible

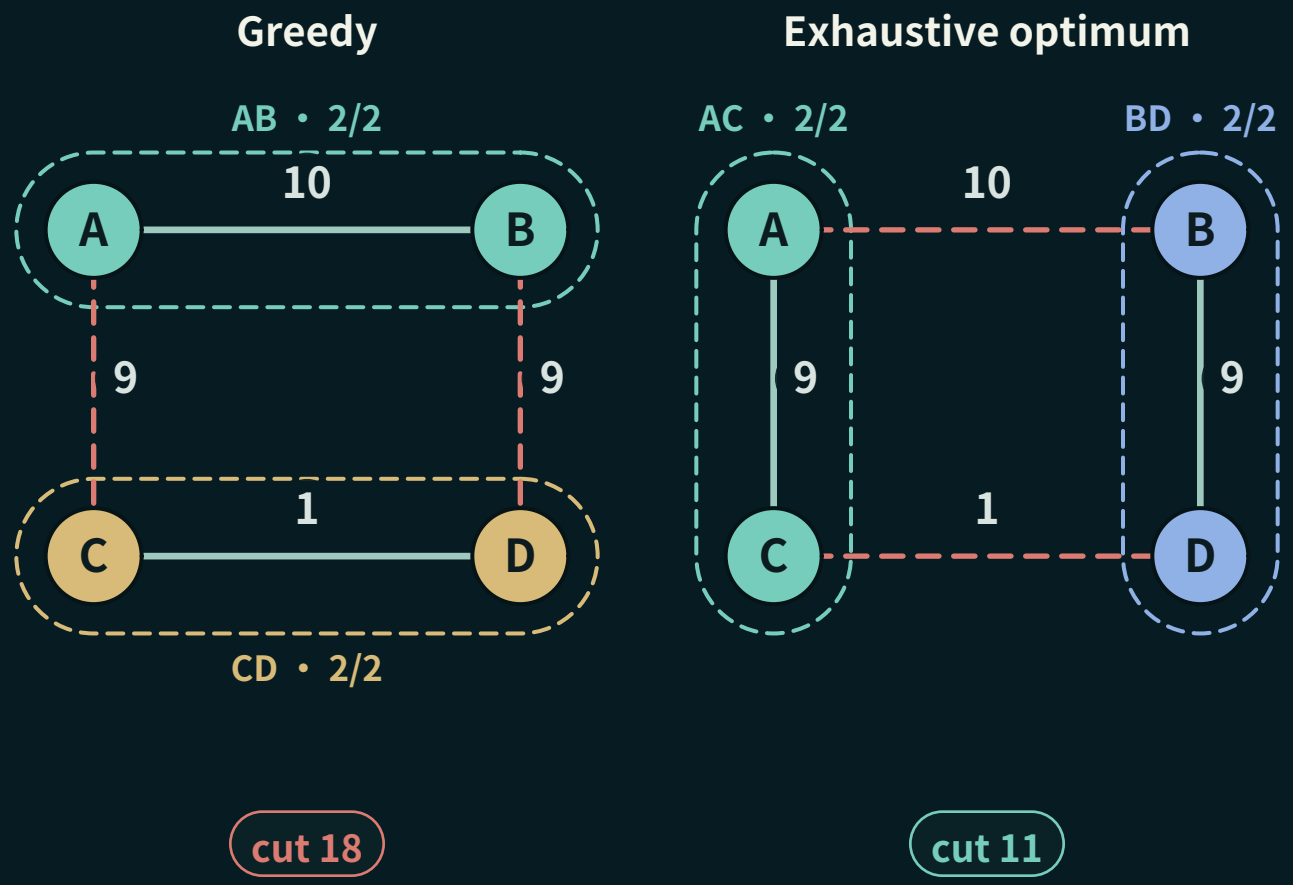
Feasible, but greedy is stuck

- greedy_trap (original author): A and B size 2; C and D size 1; capacity 3
- Bound 6 ÷ 3 = 2 does not rule out 2 clusters
- Greedy merges C–D (10) first; CD+A and CD+B are both 4 > 3
- Stops at 3 clusters, yet exhaustive search finds AC | BD (3, 3), cut 10
- Stopped ≠ no solution



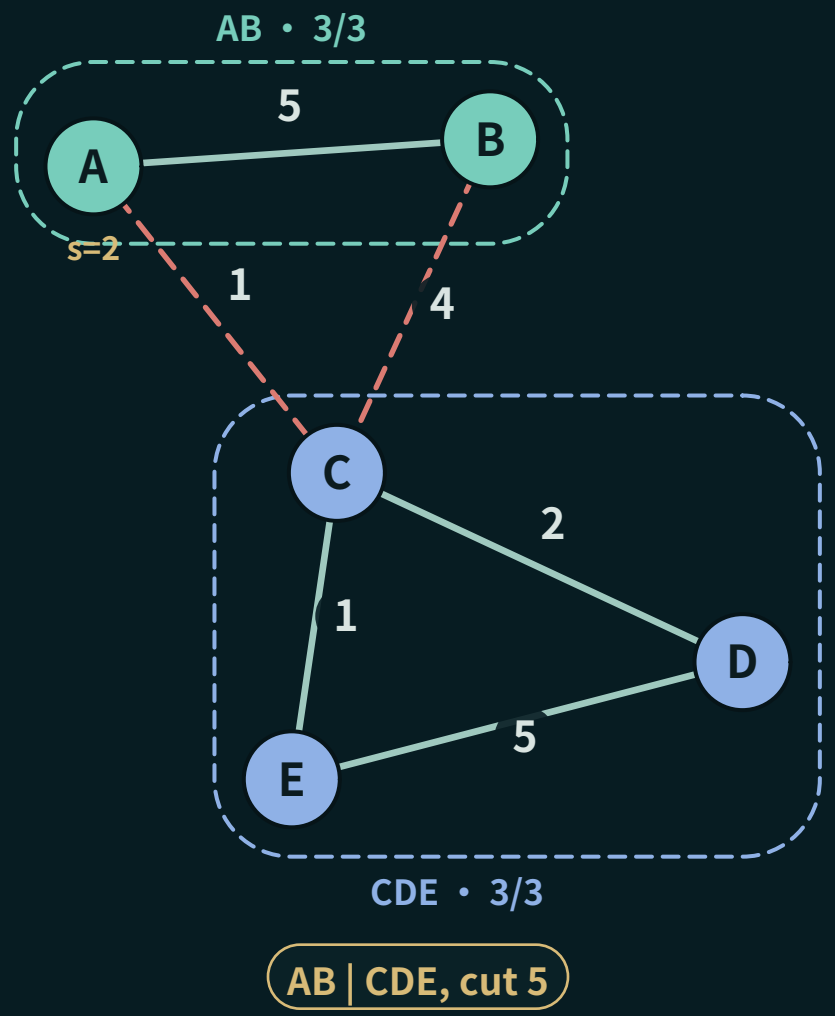
Legal, but not optimal

- suboptimal (original author): A–B 10, A–C 9, B–D 9, C–D 1
- Greedy: AB | CD, both 9s cut, cut 18
- Exhaustive: AC | BD, cuts 10 and 1, cut 11
- Both are legal and both reach K; only the quality differs



Different sizes, different endings

- Set A's size in G01 to 2, capacity 3, K = 2
- After A–B the size is 3, full; AB+C is blocked
- D–E instead, then C goes into DE
- Result AB | CDE, cut 5 (was ABC | DE, cut 3)



The pseudocode: merge, report, diagnose

- Line 1: a single node over capacity means invalid input
- Line 4: filter for legality before comparing
- Line 5: tell capacity-blocked from no-positive-affinity
- Line 7: report four items separately
- Line 8: diagnose with the lower bound plus small-graph search

```
1  clusters ← singletons; s(v) > C: invalid input
2  while #clusters > K:
3    cands ← pairs with affinity > 0
4    legal ← cands with s(P) + s(Q) ⩽ C
5    if legal is empty: stop (two reasons)
6    merge the legal pair with the least key
7  report count, legal, K reached, stop reason
8  diagnose: bound ⌈Σs / C⌉ and small search
```

Common mistakes

- Calling "legal" "done"
- Breaking the capacity to reach K
- Taking "greedy is stuck" for "impossible"
- Assuming a legal result is optimal
- Treating node count as area

Practice and limits

- Capacity is the area one region of the chip can hold
- A bound only proves impossible; possible needs a solution (as hard as bin packing)
- Exhaustive search only suits teaching graphs of $\leqslant 10$ nodes
- Greedy never backtracks: improving needs methods that swap or move back (Modules 07, 08)

Summary

Filter for legality, then take the largest; report four items separately.

Impossible (by the bound), stuck (feasible but unreachable) and not optimal (legal but worse) are three different endings.

Next module: nodes don't merge; they change "labels".